SECTION 3 · SUBSECTION 3.3
DEPENDENCY GRAPH

3.3.1 INTRODUCTION.

The dependency stage converts internal imports into a directed graph whose nodes are manifest files and whose edges point from importer to imported file. It then computes in-degree, out-degree, PageRank, membership in a cycle, and, when the graph is acyclic, a topological order that becomes the suggested reading sequence. The one idea the reader must leave with is the edge direction: an edge A → B means A imports B, so high in-degree marks a file that many others depend on (figure 3.3-1).

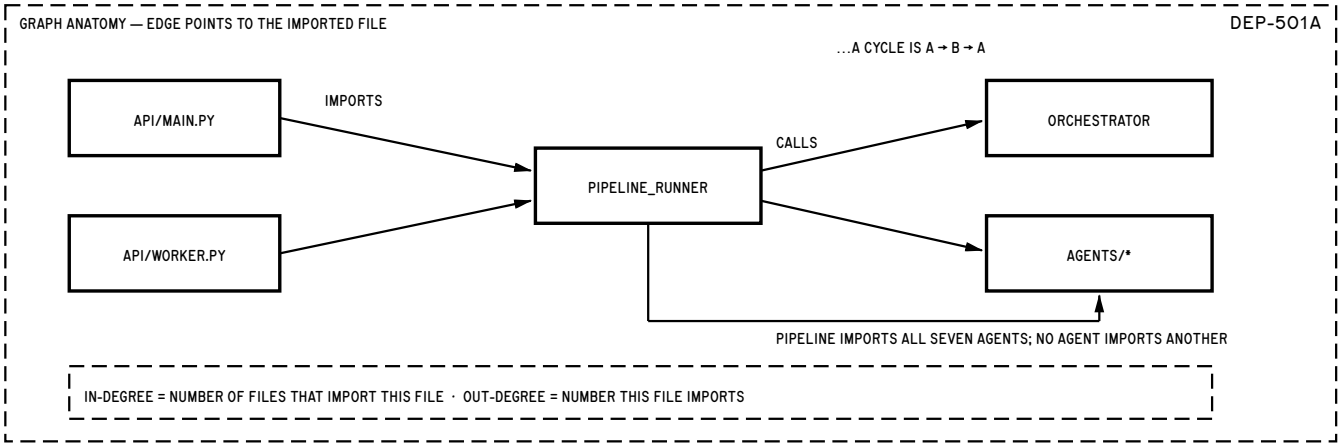


Figure 3.3-1. Edge Direction and Degree Definitions

3.3.2 FUNCTIONAL DESCRIPTION.

run adds every manifest file as a node before adding any edge, so isolated files remain visible to scoring and explanation. For each internal import it calls *resolve_import_to_paths*, which applies language-specific rules: Python absolute and relative module resolution, JavaScript extension and index resolution, Rust crate and *super* paths, C include paths, Go package directories, and Java source roots. Self-loops and targets outside the manifest are discarded. The stage then runs *nx.simple_cycles*, marks every node in a cycle, computes PageRank, and builds one statistics record per node.

3.3.2.1 Reading Order.

If the graph has no cycle, *nx.topological_sort* produces an order in which every dependency appears before its importer. If any cycle exists, the stage leaves the topological order empty rather than returning a partial order; the document then falls back to safe starting files. This is a deliberate all-or-nothing rule, not an oversight.

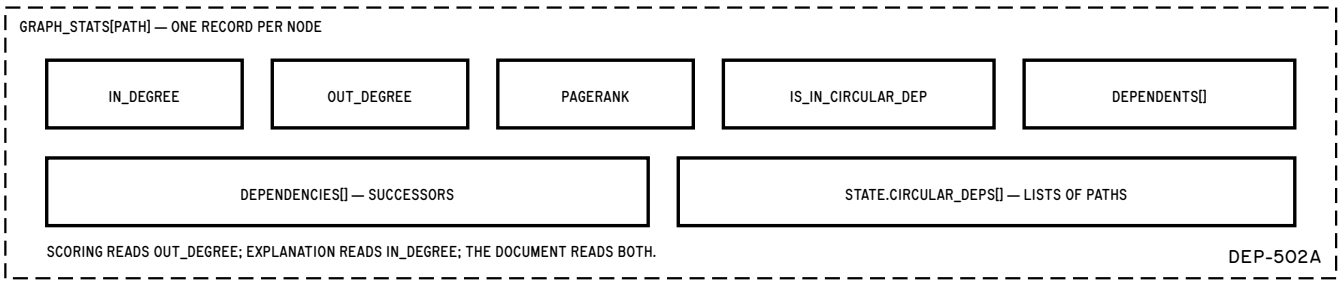


Figure 3.3-2. Statistics Record and Its Consumers

3.3.3 PERFORMANCE AND DESIGN DATA.

Table 3.3-1. Graph Construction Rules

Rule	Value
Node set	every manifest file
Edge	importer → imported
External imports	no edge
Self-loops	discarded
Missing targets	counted unresolved, dropped
Cycle algorithm	nx.simple_cycles
Rank	PageRank, 6 decimals
Topological order	empty if any cycle

CAUTION

simple_cycles enumerates every elementary cycle, and the cost can grow sharply on dense graphs. A repository with many mutually importing packages may spend most of stage 3 in cycle enumeration. The count is exact; it is not a sample.

```
GRAPH CHECK                          MAINTAINER
1 READ - node count equals manifest
2 PICK - one known import
  √ EDGE - direction is importer → target
3 CHECK - cycle list non-empty only if real
4 CONFIRM - topo empty when cycles exist
```